Chapter 2 Hypothesis Testing and Statistical Quality Control

Dennis Epple, Stephen Epple

Tepper School of Business

Copyright Dennis Epple

Statistical Decision Making (45-752)

[ ]:
```
import pandas
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import norm, t, ttest_1samp

# Jupyter "magic" command
# precision - round floats to 4 significant digits
%precision %.4f
```

[ ]: '%.4f'

1 Hypothesis Testing and Statistical Quality Control

1.0.1 Learning objectives

Some of this chapter is review of material you learned in the Probability and Statistics course. Hence, you will already be familiar with several concepts is this chapter.

After completing this chapter, you will be able to:

- Describe the key components of hypothesis testing, including null and alternative hypotheses, test statistics, p-values, and decision rules.
- Apply hypothesis testing to evaluate claims about data.
- Explain confidence interval and acceptance interval
- Plot and interpret control charts for monitoring continuous variables and binary variables.

1.1 Introduction

There is a substantial amount in the chapter. Some is review and some is new. You should not worry if it takes you more than one sitting to work through the chapter. My hope is that you will find this a useful reference source as you move forward in learning statistical decision making. The chapter covers the following three topics as well as providing coverage of some basics of using Python.

- Hypothesis testing

- Central Limit Theorem
- Statistical quality control

Please import the following data sets which we use in this chapter:

```
[ ]: PowerToGo = pandas.read_excel("data/PowerToGo.xlsx")
     ReadyPlus = pandas.read_excel("data/ReadyPlus.xlsx")
     Battery_Data = pandas.read_excel("data/Battery_Data.xlsx")
     Theta_data = pandas.read_excel("data/Theta_data.xlsx")
     Theta_New = pandas.read_excel("data/Theta_new.xlsx")
     Defects_data = pandas.read_excel("data/Defects_data.xlsx")
```

1.2 Hypothesis Testing

Hypothesis testing is used extensively in statistical analysis for making decisions. The motivation for the first application we will study in this chapter is the following. For quite some time, Duracell marketed its batteries with the claim that its batteries last up to 10 times longer than ordinary zinc carbon batteries. A competitor brought a complaint to the Advertising Authority of South Africa challenging Duracell's claim. The Agency reviewed the case and made the decision that "…the claims and advertising in question communicates the claims in a misleading manner…" The agency ruled that "The claims to 'last up to 10 times longer' must be withdrawn" http://asasa.org.za/rulings/duracell-10timeslonger_eveready_2017-6182f

As a future manager, you will want to be able to back up claims about your company's products with compelling evidence. Statistics provides you with the methodology to be able to do that. How might a company that produces and sells batteries back up its claims? Let's work through the following case.

PowerToGo (PTG) is a company that manufactures batteries. PTG believes that its D Cell batteries will last about 70 hours in continuous usage at a typical rate of power drain of 200 milliamps. How might PTG conduct an experiment to investigate its belief that typical battery life is 70 hours?

What is the null hypothesis that PTG wishes to test?

- The null hypothesis is that μ=70.
- The alternative hypothesis is that $\mu \neq 70$.

The null hypothesis value is often denoted μ_0. We will use this notation below.

The manager has decided to use a significance level α=.05. This standard is widely used in practice.

The manager selects a sample of 30 PowerToGo size D batteries as follows. Each day for 30 business

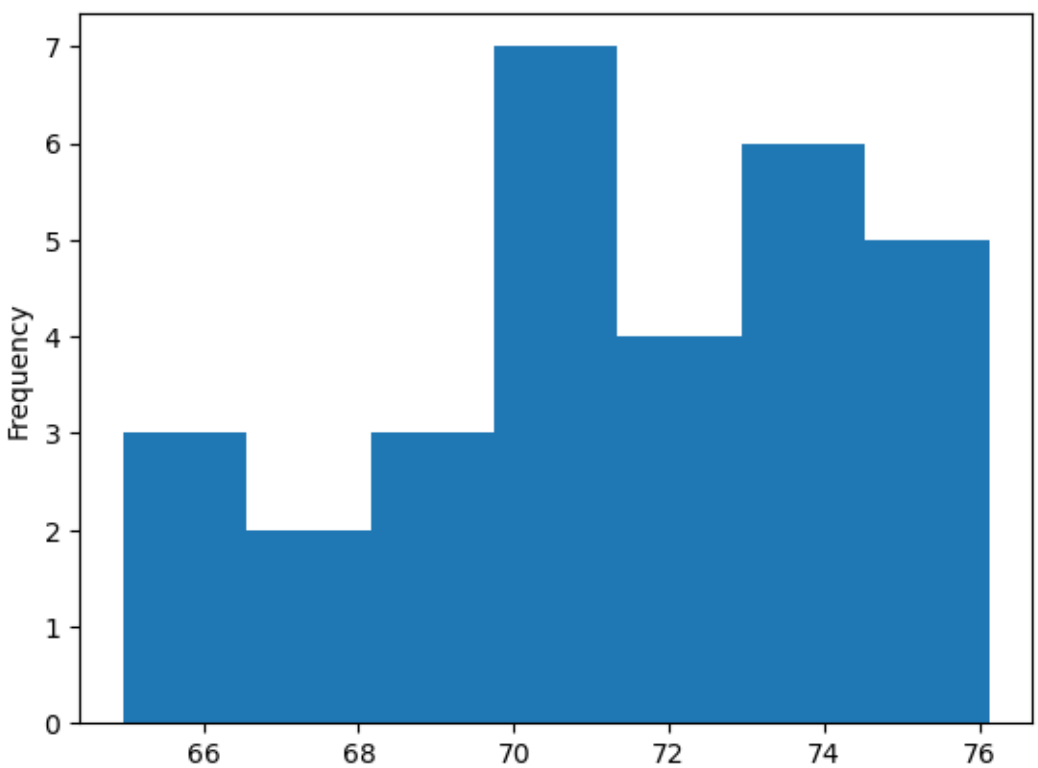
days, she selects a single battery from the production line.[1] On each of those days, she randomly chooses the time of the day to select the battery. When the 30 batteries have been obtained, she has her engineering staff connect each battery to a light that has a power drain typical of normal battery usage (200 milliamps). She then has these batteries and their connected lights placed on a large table in a conference room. A digital clock is placed on the table that shows time in minutes. She has a video camera set to run continuously to record the 30 lights on the table. The lights connected to the batteries are then switched on and left on until all of the lights go out. When the last of the lights has stopped shining, she has her assistant view the video and record the time at which each light went out. This video will be posted on the company web page so that prospective customers can see for themselves how the test was conducted.

It would have been simpler and quicker if the manager had gone to the assembly line and selected the next 30 batteries to come off the production line. Why didn't she do that?

The results of the experiment are in the PowerToGo Excel file with variable named Hours. These are data that I made up to illustrate the analytic approach. It is always a good idea to "look at your data". In this application, a histogram is a good way to do that. We see that battery lives in the sample ranged between 64 and 78 hours. There are no outliers or other visible reasons for concern in this histogram.

[ ]:
```
PowerToGo["Hours"].plot.hist(bins=7)
```

[ ]:
```
<Axes: ylabel='Frequency'>
```



[1] If you wish to see a battery production line, follow this link. https://www.youtube.com/watch?v=ksxSOwA933M

Using the following command, the manager calculates the sample mean from her data. We see that the average life of the 30 batteries was 70.3 hours. The manager will use this in her hypothesis test.

[ ]:
```
PowerToGo["Hours"].mean()
```

[ ]:
```
np.float64(71.30575489873489)
```

Before seeing how the manager conducts the hypothesis, we will review statistical concepts that provide the foundation for hypothesis testing.

1.3 The Law of Large Numbers

This course is focused on applying statistical methods to analyze data and make decisions. Here we discuss some key concepts, highlighted in bold. Consider the manager's investigation of battery life. In this application, the company is producing a size D battery with a given design. The **population** of interest is all size D batteries of this design that the company has produced and will produce. Let this population have mean battery life μ, and battery life variance σ^2. Both μ and σ^2 are unknown. The form of the distribution of battery lives is also unknown; we don't know whether the distribution of battery lives is uniform or normal or some other distribution.

Suppose we have a simple random sample of n batteries, and we measure the battery life of each. Let the measured lives of the batteries be denoted x_1, x_2,...x_n. A **simple random sample** is one for which, on each draw, every member of the population has an equal chance of being drawn. Let $\overline{X}$ be the **sample mean** of such a simple random sample: $\overline{X} = \sum_i^n \frac{x_i}{n}$ The sample mean is itself random. You know from your probability course that the mean of a simple random sample is an unbiased estimator of the population mean. This unbiasedness property is expressed in terms of expected value as: $E(\overline{X}) = \mu$.

We also know that the variance of $\overline{X}$ is σ^2/n. This tells us that, as sample size n increases, the variance of the sample mean $\overline{X}$ decreases. This is the **Law of Large Numbers**. The Law of Large Numbers states that, as sample size increases, the sample mean $\overline{X}$ becomes an increasingly precise estimate of μ.

A further thought regarding the definition of a simple random sample: As you think about the above definition of a simple random sample, you may say to yourself: "Wait a minute. There is no way that the manager can select from the entire population because it is not possible to select a battery that hasn't yet been produced." Right you are. The analysis is implicitly assuming that the sample taken over the course of 30 working days is representative of the batteries that will be produced in the future given the production procedures the company is currently employing. The experiment would be repeated regularly to investigate whether batteries being produced continue to meet the company standard.[2]

To lay the groundwork for conducting the hypothesis test, we next review the normal distribution and then the t-distribution.

[2] Batteries also undergo statistical quality control at the time of production to investigate whether they are assembled properly and deliver the intended power.

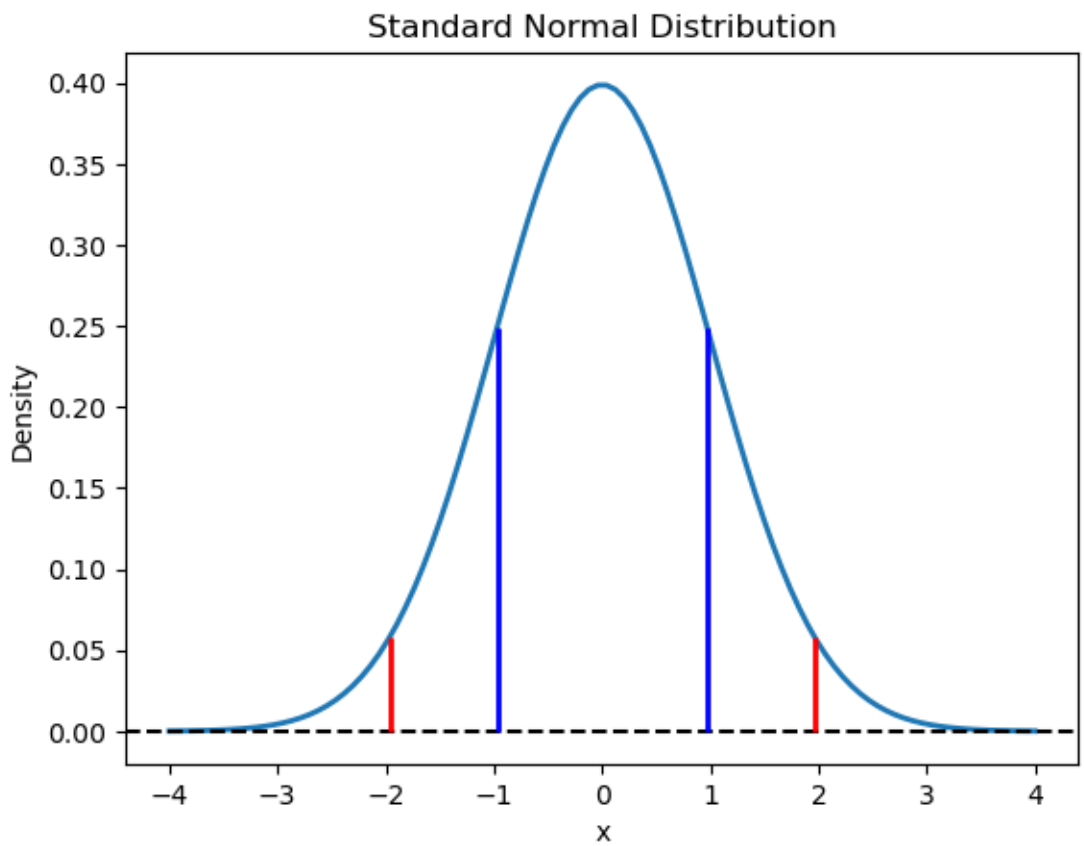
1.4 The Normal Distribution

The normal distribution is a bell-shaped curve. As with any probability distribution, the area under the curve is equal to one. 95% of the area under the standard normal distribution lies in the interval [-1.96,1.96]. This is a useful rule of thumb for you to remember: **To a very close approximation, 95% of the area under the normal distribution lies within two standard deviations of the sample mean.** This interval is demarcated by the outer vertical lines shown in red in the graph below.

Two thirds of the area under the normal distribution lies in the interval [-.97,.97]. This interval is demarcated by the inner vertical lines shown in blue in the graph below. Thus, approximately two thirds of the area under the normal distribution lies with one standard deviation of the mean.

Note: If you are viewing a printed version without color, the blue lines are the inner vertical lines and the red are the outer vertical lines.

[ ]:
```
# Display the normal distribution

# Generate 100 values equally spaced from -4 to 4
x = np.linspace(-4, 4, 100)

# Compute the probability density function (PDF) values for the standard normal␣
↪distribution
hx = norm.pdf(x)

# Create the plot
plt.plot(x, hx, linewidth=2, label="Standard Normal Distribution")

# Add labels and title
plt.xlabel("x")
plt.ylabel("Density")
plt.title("Standard Normal Distribution")

# Add horizontal line at y=0
plt.axhline(0, linestyle="--", color="black")

# Add vertical lines at x = ±1.96 and x = ±0.97 with different colors
plt.vlines([-1.96, 1.96], 0, norm.pdf(1.96), colors="red", linewidth=2)
plt.vlines([-0.97, 0.97], 0, norm.pdf(0.97), colors="blue", linewidth=2)

# Display the plot
plt.show()
```



The formula for the normal distribution with mean μ and standard deviation σ is the following.

$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}}$

Above, we plotted the standard normal distribution which has mean $\mu = 0$ and standard deviation $\sigma = 1$.

Using Python to find information about the normal distribution. Suppose a random variable x has a normal distribution with a mean of 2 and a standard deviation of 3. How much area under the distribution lies to the left of x=1.7? We find the answer using the norm.cdf() command as follows. The second and third entries in the norm.cdf() command are respectively the mean and standard deviation of the distribution. We see that .46 of the area under the distribution is to the left of x=1.7.

[ ]:
```
norm.cdf(1.7, 2, 3)
```

[ ]:
```
np.float64(0.460172162722971)
```

Note: The above command calculates a probability of obtaining outcome less than 1.7 when we take a random draw from a normal distribution with mean 2 and standard deviation 3. For the same distribution, what is the value of x such that 60 percent of the area

under the distribution lies to the left of x? For this, we use the norm.ppf() command. Again, the second and third entries are the mean and standard deviation of the distribution.

[ ]:
```
norm.ppf(0.6, 2, 3)
```

[ ]:
```
np.float64(2.7600413094073994)
```

Note: The above command calculates of a quantile–a point on the horizontal axis of the distribution. We find that a normal distribution with mean 2 and standard deviation 3 has 60% of its area to the left of quantile 2.76. For practice with the above two commands, answer the following.

- What proportion of the area lies to the left of 1.9 in the above distribution?
- What is the value of x such that 75% of the area is to the left of that value?

1.5 The Central Limit Theorem

When we do statistical analysis, we rely extensively on the Central Limit Theorem. This is an astonishing result and surely the most important result in all of statistics. You studied this in the Probability and Statistics course. When I first was taught this theorem, I didn't really understand it. Eventually, I caught on. You may have a similar experience. So don't worry if you find this section a bit difficult to follow on the first (or second) pass. At some point, you will get the "Aha!" experience.

The Central Limit Theorem extends the Law of Large Numbers in a very important way. We don't know μ, we don't know σ^2, and we don't know the distribution of battery lives. We wish to test a hypothesis about μ. So, how is it that we can conduct a hypothesis test if we don't know the form of distribution of battery life? That is where the **Central Limit Theorem** comes into play. If sample size n is "large enough" then $\overline{X}$ can be treated as a draw from a normal distributed with mean μ and standard deviation $\sigma/\sqrt{n}$. This is true even if the distribution from which the sample is drawn is not normal.

How large is large enough? If we need a sample of a million batteries in order to employ the Central Limit Theorem, the theorem would be useless. Fortunately, and rather remarkably, the sample size generally doesn't need to be especially large. The sample size that is needed depends to some extent on the distribution from which the sample is drawn. If that distribution is not too skewed, the Central Limit Theorem can typically be applied if the sample size, n, is 30 or more. This is a widely used rule of thumb for applying the Central Limit Theorem. For highly skewed distributions, bigger samples are needed for the approximation to be valid. Above we saw that a histogram of the observations for our PowerToGo data is not noticeably skewed. This suggests that we can comfortably use the n=30 rule of thumb.

So far, so good. But when we do our hypothesis test, we encounter a second issue. Recall that the variance of $\overline{X}$ is σ^2/n. We do not know σ^2. We estimate σ^2 using the **sample variance** $s^2 = \sum_{i=1}^{n} \frac{(X_i - \overline{X})^2}{n-1}$. Why divide by $(n-1)$? To understand why, suppose we knew the population mean μ. We would then calculate the sample variance using $\sum_{i=1}^{n} \frac{(X_i - \mu)^2}{n}$. In practice, of course, we do not know the population mean, and we must use the sample mean when calculating the sample variance. This adds an additional element of uncertainty. Dividing by $(n-1)$ when we estimate the population variance adjusts for the fact that we don't know the population mean and

must estimate it. When we have a simple random sample, the sample variance s^2 is an unbiased estimator of σ^2: $E(s^2) = \sigma^2$.

We do hypothesis testing using s^2 because we do not know σ^2. This is where the t-distribution comes into play. The t-distribution takes account of the additional uncertainty that arises when we use s^2 in place of the unknown σ^2.

For your review of our discussion thus far:

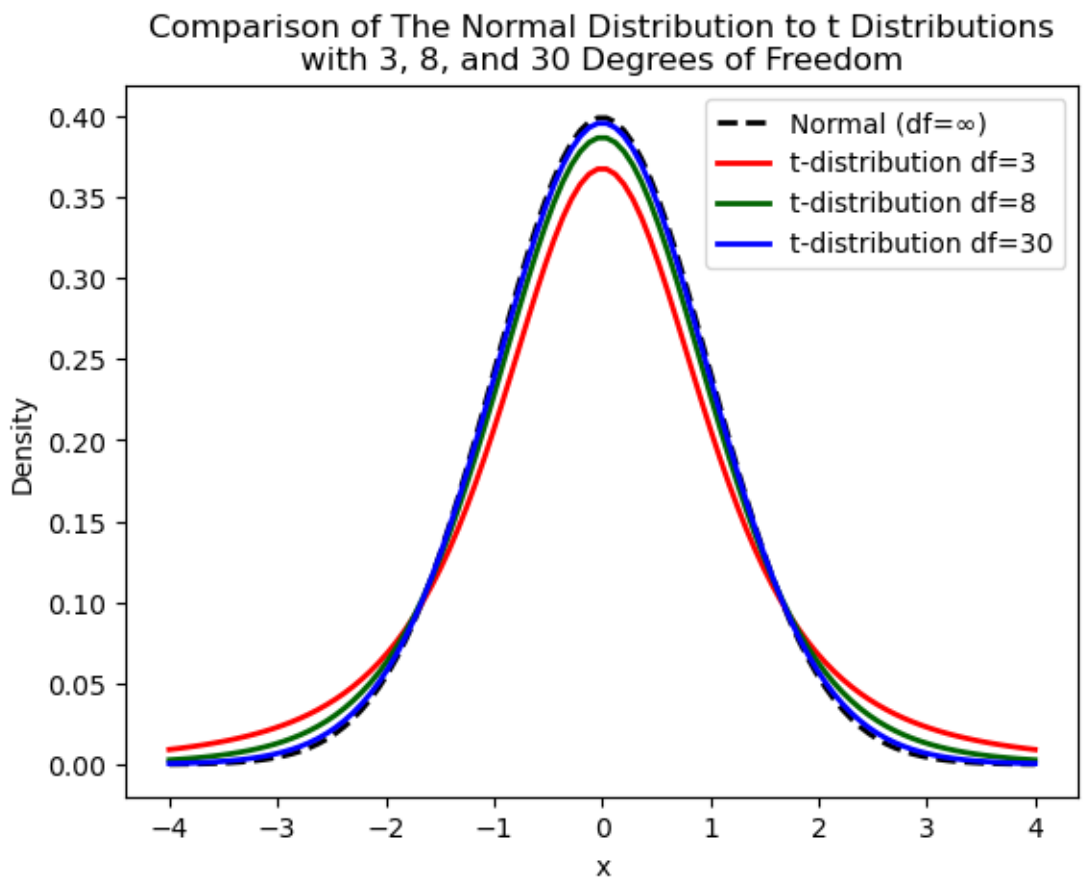
- A simple random sample of 10 is drawn from a population with a population variance equal to 5. What is the variance of the sample mean?
- A simple random sample of 30 is drawn from a population with a population variance equal to 5. What is the variance of the sample mean?
- A Tepper alum was hired by a company that provides cell phone service. The company purchases a huge number of phones for distribution to customers who purchase the company's cell phone service. Our alum discovered the following. When a large shipment of phones arrives, the company takes a sample of 30 "off the top" of the shipment and tests them. If those phones work well, then the company accepts the shipment. Does that strike you as a good way to draw a random sample of phones to test? Hint: Is each phone equally likely to be chosen?
- Each phone purchased by the company has a distinct serial number. When a shipment of phones is received, a spreadsheet with the list of serial numbers is provided to the company. Our alum put in place a new procedure to select phones to test. The serial numbers for phones in a shipment are sorted in random order. Then the 30 phones at the top of the sorted list are tested. Thus, every phone has an equal chance of being selected. Does this strike you as a good way to choose a sample of phones to test? The Appendix to this chapter provides more detail on the procedure for sorting items in random order.

1.6 The t Distribution

We use the t distribution for hypothesis testing because it accounts for the fact that we do not know σ^2. Let's study the t-distribution before doing the hypothesis test. The graph below shows the normal distribution as a dashed curve. The graph also shows three t-distributions. The t-distribution in red has 3 degrees of freedom (df), the distribution in blue has 8 df, and the distribution in green has 30 df. From this comparison, you can see that the lower the degrees of freedom, the thicker are the tails and the lower the middle of the t-distribution. We see that all of the t-distributions have a lower middle and thicker tails than the normal distribution. The reason is as follows. If we do a hypothesis test with the normal distribution, we are treating the population mean as unknown, but we are treating the population standard deviation as known. In practice, of course, we do not know either and we must estimate both. The need to estimate the standard deviation gives rise to additional uncertainty compared to the normal distribution. The smaller the sample size, the greater the effect of this additional uncertainty. This is why the t-distribution was developed. When we test a hypothesis about a population mean with a sample of size n, the degrees of freedom equals $n-1$. The larger the sample size, the closer the t-distribution becomes to the normal distribution. You can see from the figure below that, with df=30, there is relatively little difference between the t and the normal.

[ ]:
```
# Set up the x values (range from -4 to 4)
x = np.linspace(-4, 4, 100)
```

```python
# Calculate the standard normal distribution
hx = norm.pdf(x)

# Degrees of freedom for the t-distributions
degf = [3, 8, 30]
colors = ["red", "darkgreen", "blue"]
labels = [f"df={df}" for df in degf]

# Plot the normal distribution (dashed line)
plt.plot(x, hx, "k--", lw=2, label="Normal (df=∞)")

# Plot the t-distributions
for df, color in zip(degf, colors):
    plt.plot(x, t.pdf(x, df), lw=2, color=color, label=f"t-distribution␣
 ↪df={df}")

# Labels and title
plt.xlabel("x")
plt.ylabel("Density")
plt.title(
    "Comparison of The Normal Distribution to t Distributions\nwith 3, 8, and␣
 ↪30 Degrees of Freedom"
)

# Add a legend
plt.legend()

# Show the plot
plt.show()
```



In short, the graph shows that a t-distribution with low df is flatter and has thicker tails than a t-distribution with high df. We use symbol ν to denote degrees of freedom. We use $t_{\alpha/2,\nu}$ to denote the t value with degrees of freedom=ν and $\alpha/2$ of the area under each tail of the distribution. When we do hypothesis testing with significance level $\alpha/2$, we call this $t_{\alpha/2,\nu}$ the "critical t value".

The t-distribution has an interesting history. I thought you might be interested in this brief background on the development of the t-distribution. It was named the "t" distribution as a shorthand for test distribution. Perhaps not the most creative of names, but the name has stuck.

"**William Sealy Gosset** (1876-1937) was born in Canterbury, England. He studied at Winchester, then at Oxford. Upon graduation, he joined Arthur Guinness and Son, a Dublin brewery. Early on, Gosset saw a need for careful scientific analyses of a variety of processes, from barley production to yeast fermentation, all of which profoundly affected the quality of the brewery's final product, beer. His firm sent him to study under Karl Pearson (Biography 14.1) at the University College, London. At the time, the theory of estimation based on large samples had been fully worked out, but Gosset noticed a void with respect to small-sample estimation theory. Small samples, however, were typical of Gosset's work at the brewery. So Gosset developed the theory himself. In a now famous 1908 paper, The *Probable Error of a Mean*, he noted that s is an erratic estimator of σ when n is small; hence, customary measures of the precision of estimates were invalid for small samples and unknown σ. His paper presented the sampling distribution of a statistic now known as *Student's t* and introduced small-sample estimation by means of the t-distribution family. It is difficult to overestimate the importance of this achievement. It has proven fundamental to statistical inference as it exists today, not only in the realm of estimation, but also in hypothesis testing and the analysis of variance." Source: http://www.swlearning.com/quant/kohler/stat/biographical_sketches/bio12.1.html

"Another researcher at Guinness had previously published a paper containing trade secrets of the Guinness brewery. To prevent further disclosure of confidential information, Guinness prohibited its employees from publishing any papers regardless of the contained information. This meant that Gosset was unable to publish his works under his own name. He therefore used the pseudonym *Student* for his publications to avoid their detection by his employer. Thus his most famous achievement is now referred to as the Student t-distribution, which might otherwise have been the Gosset t-distribution." Source: http://en.wikipedia.org/wiki/William_Sealey_Gosset

Using Python to find information about the t distribution. To help you get a feel for the t-distribution, let's investigate the effect of differences in degrees of freedom ν with α=.05. The commands below calculate $t_{\alpha/2,\nu}$ for the three values of ν in the graph above. For the t-distribution with ν=3, we obtain a t-value of 3.18. This tells us that 2.5% of the area under the t distribution with ν=3 lies to the left of -3.18 and 2.5% lies to the right of 3.18. For ν=8, the range decreases to [-2.31,2.31]. For ν=30, the range decreases to [-2.04,2.04]. By way of comparison we saw above that, for the normal distribution, the range is [-1.96,1.96].

[ ]:
```
v3 = t.ppf(0.975, 3)  # Three degrees of freedom
v8 = t.ppf(0.975, 8)  # Eight degrees of freedom
v36 = t.ppf(0.975, 30)  # Thirty degrees of freedom

v3, v8, v36
```

[ ]:
```
(np.float64(3.182446305284263),
 np.float64(2.306004135204166),
 np.float64(2.0422724563012373))
```

Above we calculated the .975 quantile of three different t-distributions. As with the formula for the normal, ppf tells us we are calculating a quantile.

It is customary to use df as a shorthand to denote degrees of freedom. For testing a hypothesis about a sample mean, the number of degrees of freedom is $n-1$. Hence, $n=4$ implies 3 df and $n=9$ implies 8 df. The graph above is to help you understand the effect of differences in df. As a practical matter, you would not do a hypothesis test with sample sizes so small that you would have only 3 df or 8 df.

When we use the t-distribution, we are implicitly relying on the Central Limit Theorem. Sample sizes $n = 4$ or $n = 9$ are not nearly large enough to justify invoking the Central Limit Theorem. As noted earlier, a rule of thumb of $n = 30$ is often used when the distribution of our sample of data is relatively symmetric and without large outliers.

Suppose we wish to find the area under the t-distribution to the left of point 1.2 on the horizontal axis, and we have 15 degrees of freedom. We use the following command:

[ ]:
```
t.cdf(1.2, 15)
```

[ ]:
```
np.float64(0.8756251063059888)
```

We see that 87.56% of the area under the t-distribution with 15 degrees of freedom lies to the left of 1.2.

Review and Practice:

- Using the standard normal distribution, calculate the point on the horizontal axis (quantile) such that 20% of the area under the distribution is to the left of the point.
- Suppose we use the t-distribution with 12 df and find the point on the horizontal axis (quantile) such that 20% of the area under the distribution is to the left of the point. Without doing the calculation, will this point be to the left or right of the value you just obtained for the normal distribution?
- Check your answer to the previous question by doing the calculation.
- If a random draw is taken from a normal distribution with mean 3 and standard deviation 5, what is the probability that the result will be less than 6?
- If a random draw is taken from the t-distribution with 15 df, what is the probability that the result will be in the interval [-.5,1]?

1.7 Conducting A Hypothesis Test about A Population Mean

We now detail how the manager conducts the hypothesis test regarding battery life. Recall that the manager chose significance level α=.05. In fact, we will review four equivalent ways she might conduct the hypothesis test.

1. Calculate a 95% confidence interval. If μ_0 is not in the confidence interval, reject the null hypothesis. If μ_0 is in the confidence interval, don't reject the null hypothesis.
2. Compare the critical t-value to the t-value calculated from the data. If the calculated t-value is greater in absolute value than the critical t-value, reject the null hypothesis. Otherwise, don't reject.
3. Calculate the p-value. If the p-value is less than α, reject the null hypothesis. Otherwise don't reject.
4. Calculate the acceptance interval. If $\overline{X}$ is not in the acceptance interval, reject the null hypothesis. Otherwise, don't reject.

For any of the four approaches, the manager will need to calculate the sample mean $\overline{X}$ and the sample standard deviation, s. Recall that the formula for the sample variance is: $s^2 = \sum_{i=1}^{n} \frac{(X_i - \overline{X})^2}{n-1}$. We take the square root to obtain the sample standard deviation s. The manager calculates the sample mean and sample standard deviation using the following Python commands:

[ ]:
```
PowerToGo["Hours"].mean()
```

[ ]: np.float64(71.30575489873489)

[ ]:
```
PowerToGo["Hours"].std()
```

[ ]: 3.2119

1.7.1 Method 1: Confidence Interval

Let's now review how the confidence interval is calculated. The confidence interval is:

$[\overline{X} - t_{\alpha/2,\nu}\frac{s}{\sqrt{n}}, \overline{X} + t_{\alpha/2,\nu}\frac{s}{\sqrt{n}}]$

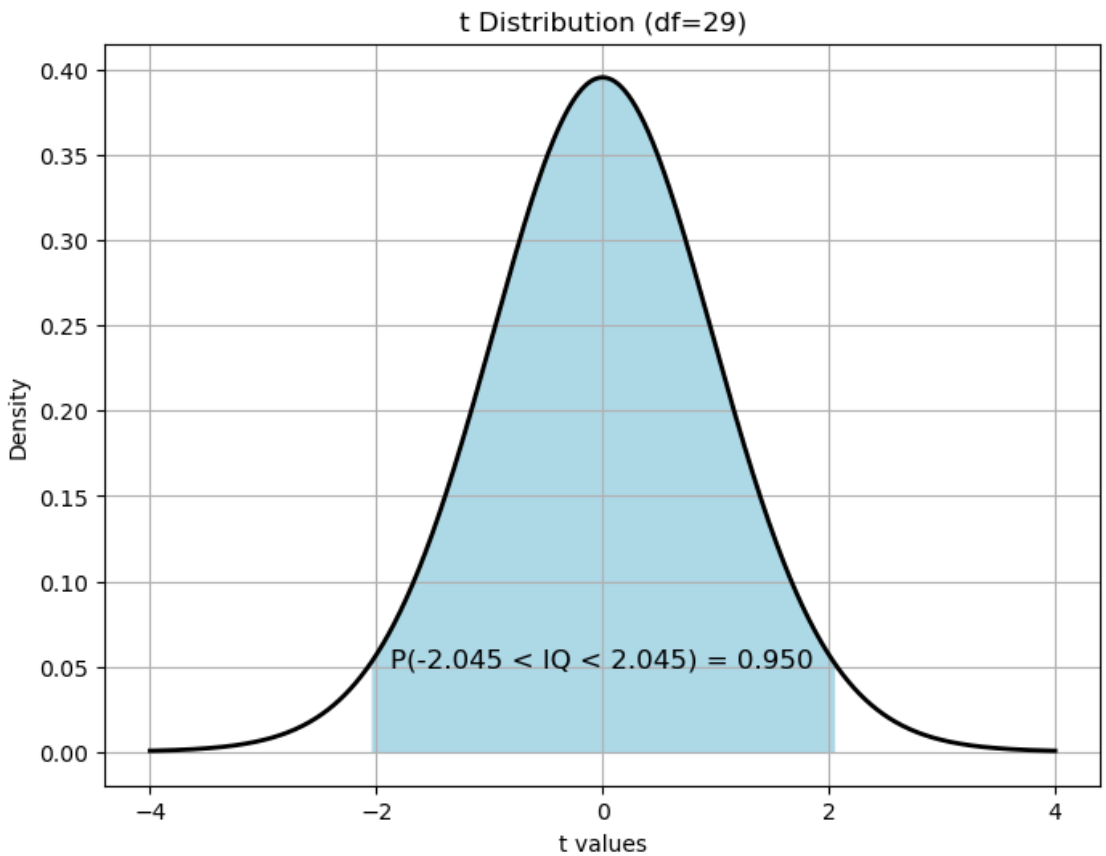
As we see from this formula, we need to find the critical t-value for our 5% significance level. You can verify that our sample has 30 data points. Hence we have $\nu = 29$ degrees of freedom. The manager calculates qt as shown below. The result tells us that, for the t-distribution with 29 degrees of freedom, 95% of the area under the distribution lies in the interval [-2.045,2.045]. This is illustrated in the plot below. The vertical lines denote the bounds of this 95% interval.

[ ]:
```
t.ppf(0.975, 29)
```

[ ]: np.float64(2.045229642132703)

[ ]:
```
# Set degrees of freedom and the bounds for the shaded region
df = 29
lb = -2.045
ub = 2.045

# Generate x values from -4 to 4
x = np.linspace(-4, 4, 1000)

# Compute the t-distribution values
hx = t.pdf(x, df)

# Create the plot
plt.figure(figsize=(8, 6))

# Plot the t-distribution curve (without axes initially)
plt.plot(x, hx, "k-", lw=2)

# Highlight the area between lb and ub
x_fill = x[(x >= lb) & (x <= ub)]  # Values between lb and ub
y_fill = hx[(x >= lb) & (x <= ub)]  # Corresponding density values

# Fill the area under the curve
plt.fill_between(x_fill, y_fill, color="lightblue", label=f"P({lb} < IQ <␣
↪{ub})")

# Calculate the probability in the shaded region
area = t.cdf(ub, df) - t.cdf(lb, df)
```

```python
# Annotate the probability on the plot
plt.text(
    0,
    0.05,
    f"P({lb} < IQ < {ub}) = {area:.3f}",
    horizontalalignment="center",
    fontsize=12,
)

# Set plot labels and title
plt.xlabel("t values")
plt.ylabel("Density")
plt.title("t Distribution (df=29)")

# Set the x-axis ticks
plt.xticks(np.arange(-4, 5, 2))

# Add grid and make it visually clearer
plt.grid(True)

# Show the plot
plt.show()
```



We now have the information to calculate our 95% confidence interval: $[\overline{X} - t_{\alpha/2,\nu}\frac{s}{\sqrt{n}}, \overline{X} + t_{\alpha/2,\nu}\frac{s}{\sqrt{n}}] = [71.31 - 2.045 * 3.21/\sqrt{30}, 71.31 + 2.045 * 3.21/\sqrt{30}]$
We obtain [70.11,72.51].

Be sure that you know where we obtain all of the numbers used in this calculation. The manager observes that the hypothesized null value, μ_0=70, is not in this confidence interval. Hence, the manager rejects the null hypothesis. This is good news. The batteries perform even better than expected.

In the above calculation, we use the expression $\frac{s}{\sqrt{n}}$. This expression is called the standard error of the sample mean. Going forward, we will denote this standard error as $s_{\overline{x}}$, that is, $s_{\overline{x}} = \frac{s}{\sqrt{n}}$. With this notation, we can rewrite the confidence interval for the sample mean as:
$[\overline{X} - t_{\alpha/2,\nu}s_{\overline{x}}, \overline{X} + t_{\alpha/2,\nu}s_{\overline{x}}]$.
Above, we found $\overline{X} = 71.306$ and $s = 3.212$. Thus, for our application, the standard error of $\overline{X}$ is $s_{\overline{x}} = \frac{3.21}{\sqrt{30}}$=.586.

1.7.2 Method 2: Comparing Calculated t-value to Critical t-value

The manager calculates the t-statistic from the data using $t_{calc} = \frac{\overline{X}-\mu_0}{s_{\overline{x}}}$. Using $\overline{X} = 71.306$ and $s_{\overline{x}} = .586$, the manager obtains: $t = \frac{71.31-70}{.586} = 2.227$. In our analysis for Method 1, we found the critical t-value $t_{\alpha/2,\nu} = 2.045$. Hence, the manager rejects the null hypothesis because the

calculated t-value is greater in absolute value than the critical t-value: 2.227>2.045.

Above, I have used t_{calc} to denote the t-value calculated from the data and $t_{\alpha/2,\nu}$ to denote the critical t-value for our chosen value of α. I will adhere to this notational convention going forward.

1.7.3 Method 3: Comparing alpha to p-value

The manager uses the following command to calculate the two-tailed p-value for the hypothesis test. In this command, it is always convenient to enter the negative of the t-statistic. The command then calculates the area under the lower tail and multiplies by 2 to give the area under both tails.

[ ]:
```
2 * t.cdf(
    -2.227, 29
)  # Entries are negative of t-value and the number of degrees of freedom.
```

[ ]:
```
np.float64(0.03387248813323582)
```

We see that the manager obtained a p-value=.034. How do we interpret this value? Above we found that the sample mean number of hours of battery life was 71.31 hours. The manager's null hypothesis is μ=70 hours. Hence, the absolute difference between the sample mean and the null hypothesis is |71.31-70|=|1.31|. The p-value tells us there is a 3.4% probability of obtaining a difference as large or larger than |1.31| if the null hypothesis is true. Thus, if the population mean battery life is really 70 hours, it would be exceedingly unlikely that the manager would obtain a mean of 71.31 in a sample of 30 batteries. When you first encounter the p-value, you may find the concept a bit hard to grasp. Stick with it and you will soon get it. The p-value is by far the most widely used of the three approaches to hypothesis testing.
So be sure you remember: **If the p-value is less than α, you reject the null hypothesis**.

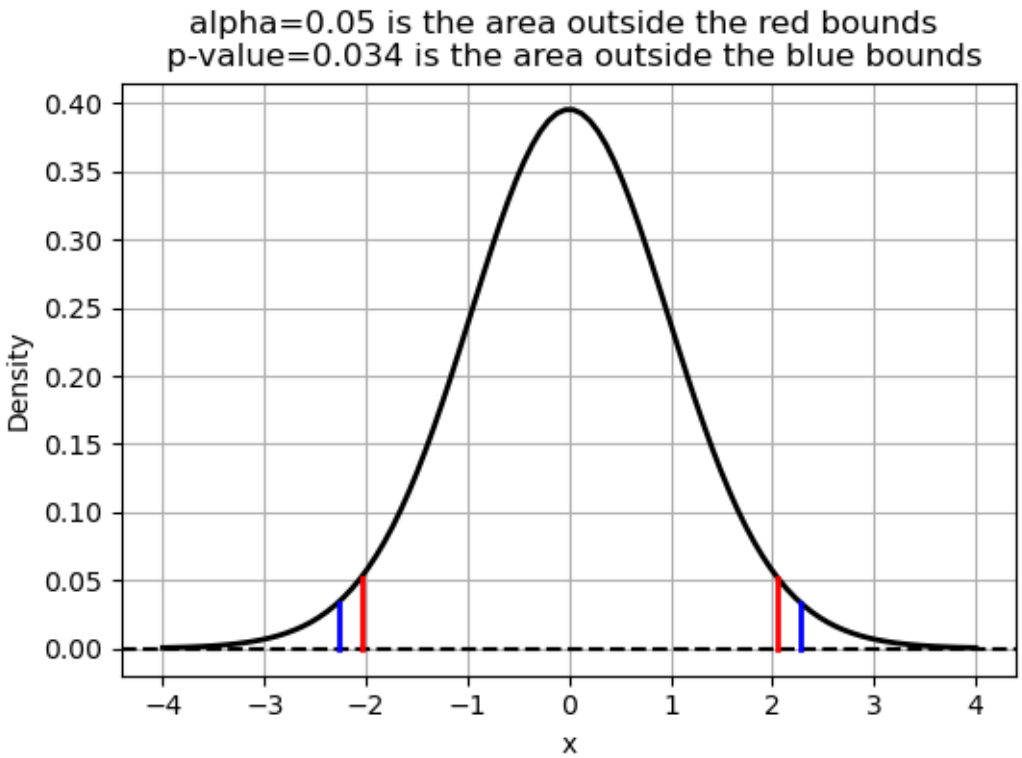
The graph below illustrates the hypothesis test. The vertical red lines are drawn at -2.045 and +2.045. These values are from our calculation of the critical t-value above, $t_{\alpha/2,\nu} = 2.045$. The area under the distribution outside these bounds is 5% of the total, with 2.5% in each tail.

The vertical blue lines are drawn at the following points on the horizontal axis -2.227 and +2.227. These are based on the t-statistic we calculated above from the data, $t_{calc} = 2.227$. The area under the distribution outside these bounds is the **p-value**. The p-value is 3.4% of the total area, with 1.7% in each tail. We see that the p-value<α. If the null hypothesis were true, the result we obtained from our data would occur with probability less than 5%. Hence, we conclude that the null hypothesis is false. As noted above, this is good news for the manager.

Note: If you are viewing a printed version without color, the blue lines are the outer vertical lines and the red are the inner vertical lines.

[ ]:
```
# Set degrees of freedom
df = 29

# Define the range of x values
x = np.linspace(-4, 4, 100)

# Compute the t-distribution values for df=29
hx = t.pdf(x, df)
```

```python
# Create the plot
plt.figure(figsize=(6, 4))

# Plot the t-distribution curve
plt.plot(x, hx, label="t-distribution (df=29)", color="black", lw=2)

# Add a horizontal line at y=0 (dashed line)
plt.axhline(0, color="black", linestyle="--")

# Add vertical segments for the critical values (-2.045 and 2.045) for alpha=0.
 ↪05 (red)
plt.plot([-2.045, -2.045], [0, t.pdf(-2.045, df)], color="red", lw=2)
plt.plot([2.045, 2.045], [0, t.pdf(2.045, df)], color="red", lw=2)

# Add vertical segments for the critical values (-2.27 and 2.27) for p-value=0.
 ↪034 (blue)
plt.plot([-2.27, -2.27], [0, t.pdf(-2.27, df)], color="blue", lw=2)
plt.plot([2.27, 2.27], [0, t.pdf(2.27, df)], color="blue", lw=2)

# Title and labels
plt.title(
    "alpha=0.05 is the area outside the red bounds \n p-value=0.034 is the area␣
 ↪outside the blue bounds"
)
plt.xlabel("x")
plt.ylabel("Density")

# Add a grid
plt.grid(True)

# Show the plot
plt.show()
```



1.7.4 Method 4: Acceptance Interval

The acceptance interval is used for hypothesis testing less frequently than the three alternatives above, but it gives exactly the same accept/reject outcome as those methods. It is very useful for you to know how the acceptance interval is constructed. Later on, we will see how the acceptance interval is used in statistical quality control. When used for hypothesis testing, the acceptance interval is calculated as if the null hypothesis is true. The acceptance interval is defined as follows: $[\mu_0 - t_{\alpha/2,\nu} s_{\overline{x}}, \mu_0 + t_{\alpha/2,\nu} s_{\overline{x}}]$. Notice how this interval differs from the confidence interval. A confidence interval is centered about $\overline{X}$. The acceptance interval is centered around μ_0.

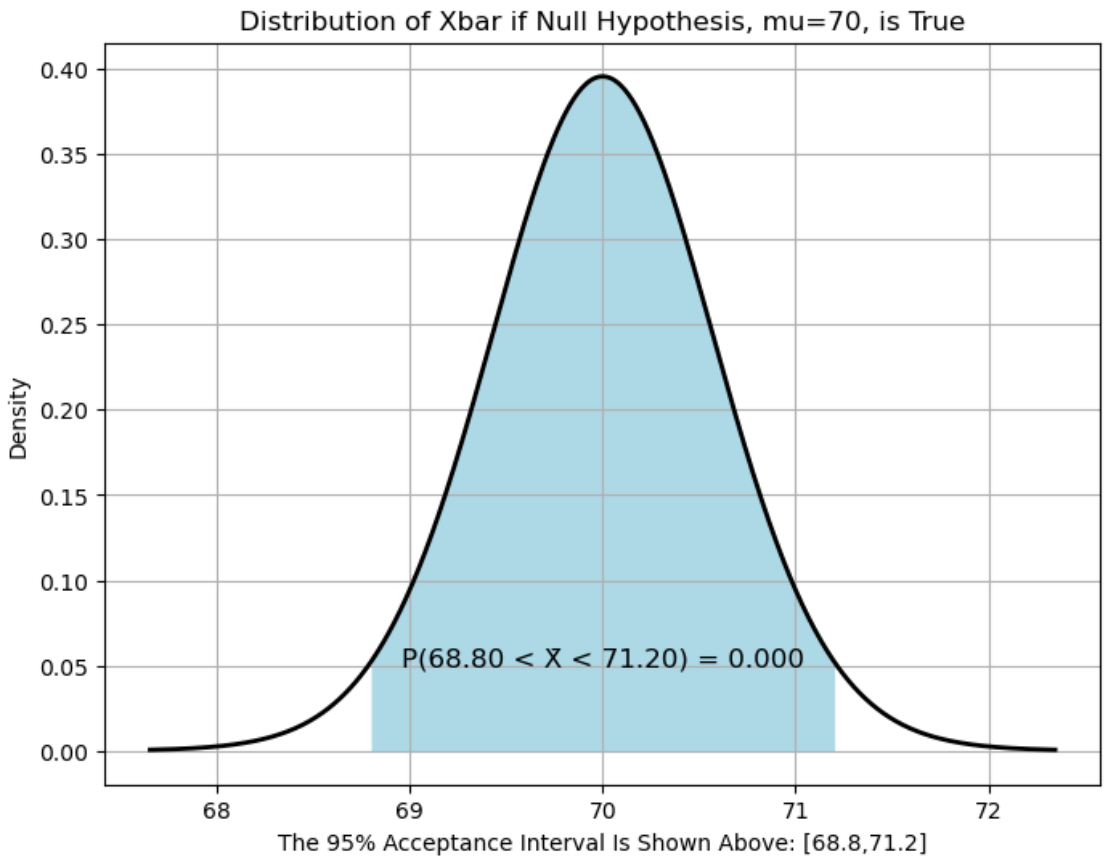
Inserting the information for our current application, we obtain $[70 - 2.045 * .586, 70 + 2.045 * .586] = [68.8, 71.2]$. If the null hypothesis is true, there is a 95% probability that our sample mean will fall in this interval. Our sample mean 71.31 is not inside the acceptance interval. Hence, we reject the null hypothesis.

The graph below illustrates our acceptance interval. The graph shows the distribution of values of $\overline{X}$ if the null hypothesis is true. The area shown in blue is 95% of the area under the distribution. The acceptance interval is the range on the horizontal axis within which 95% of the area under the distribution is contained.

[ ]:
```
df = 29
s = 3.211915
SE = round(s / np.sqrt(30), 4)  # Standard error
```

```
mu0 = 70

# Calculate the bounds for the 95% acceptance interval
lb = mu0 - 2.04523 * SE
ub = mu0 + 2.04523 * SE

# Generate x values for the plot
x = np.linspace(-4, 4, 1000)

# Compute the t-distribution values for df=29
hx = t.pdf(x, df)

# Calculate the y values for Xbar distribution
y = mu0 + x * SE

# Create the plot
plt.figure(figsize=(8, 6))

# Plot the t-distribution curve
plt.plot(y, hx, label="t-distribution (df=29)", color="black", lw=2)

# Shade the area between lb and ub (95% acceptance interval)
i = (y >= lb) & (y <= ub)  # Find where the values fall between lb and ub
plt.fill_between(
    y[i], hx[i], color="lightblue", label=f"Acceptance Interval: [{lb:.2f}, {ub:
 ↪.2f}]"
)

# Calculate the area (probability) between the bounds
area = t.cdf(ub, df) - t.cdf(lb, df)

# Add title, labels, and grid
plt.title("Distribution of Xbar if Null Hypothesis, mu=70, is True")
plt.xlabel("The 95% Acceptance Interval Is Shown Above: [68.8,71.2]")
plt.ylabel("Density")
plt.grid(True)

# Display the area (probability) on the plot
plt.text(
    70,
    0.05,
    f"P({lb:.2f} < X < {ub:.2f}) = {area:.3f}",
    horizontalalignment="center",
    fontsize=12,
)

# Show the plot
```

```
plt.show()
```



1.7.5 Testing a Hypothesis when You Have the Data

It is important for you to know how to work through the details of hypothesis testing as we did above so that you fully understand the logic and can explain the idea to colleagues who may not be familiar with statistical analysis. When you have the data, you can let the software do the work, as we do next. Notice that the command reports both the t-statistic and the p-value.

[ ]:
```
t_stat, p_value = ttest_1samp(PowerToGo["Hours"], 70)
t_stat, p_value
```

[ ]:
```
(np.float64(2.226682395972205), np.float64(0.03389602581384034))
```

We see that the t-statistic in the output is 2.23, as we calculated earlier. There are 30 data points, and, hence, 29 degrees of freedom. The p-value=.034, as we obtained previously. The 95% confidence interval is also reported. With two digits after the decimal point, the confidence interval is [70.11, 72.51], as we calculated above "by hand".

1.7.6 More on Confidence Intervals

In most situations, when we report a confidence interval, we will want to report a 95% confidence interval. However, there may be times when you want a different confidence interval. Suppose you want a 90% confidence interval. For practice, let's calculate this confidence interval. Before we do the calculation, answer for yourself the following questions. Then check your answers against our findings below.

- Will the t-statistic calculated from the data change?
- Will the p-value change?
- Will the 90% confidence interval be wider or narrower than the 95% confidence interval?

To do our calculation, we need the critical t-value for α=.10. Look at the t.ppf() command above. How would you change the command to obtain the critical t-value for α=.10?

Hopefully, you chose the command below which has 5% of the area above the upper tail, and, hence, 5% below the corresponding lower tail. For practice, modify the command by replacing .95 with .05. Then execute the command

[ ]:
```
t.ppf(0.95, 29)
```

[ ]:
```
np.float64(1.6991270265334972)
```

We obtain a critical t-value of 1.70. You now have all of the information to calculate the 90% confidence interval. Try doing the calculation yourself before looking ahead to the solution.

Answer: We calculate our 95% confidence interval as follows:
$[\overline{X} - t_{\alpha/2,\nu} s_{\overline{x}}, \overline{X} + t_{\alpha/2,\nu} s_{\overline{x}}]] = [71.31 - 1.70 * .586, 71.31 + 1.70 * .586]$
We obtain [70.31,72.31]. Let's now have Python do the calculation to obtain the 90% confidence interval and check our result. We issue the following command.

[ ]:
```
res = ttest_1samp(PowerToGo["Hours"], 70)
ci_low, ci_high = res.confidence_interval(0.9)
ci_low, ci_high
```

[ ]:
```
(np.float64(70.30936518348757), np.float64(72.30214461398221))
```

1.8 Implications for Management

We found that the manager rejects her null hypothesis that the population mean μ=70. Is this bad news for the manager? No. The estimate from her sample, 71.31 hours, is significantly *higher* than her null hypothesis, $\mu_0 = 70$ hours. Battery life is even better than expected. Hence, the company can be quite comfortable advertising that typical life of a PowerToGo battery is 70 hours. No customer will complain if actual battery life turns out to be even higher than advertised. If the null hypothesis was rejected and the sample mean was less than 70, the claim of 70 hours would not be valid.

Test your understanding: The web site below reported tests of three types of AA batteries: Duracell_Plus, Energizer_Max, Duracell_Ultra
https://mybroadband.co.za/news/energy/244152-duracell-vs-energizer-ultimate-battery-showdown.html

- Eight batteries of each type were tested. The AA size is a small battery. Battery life was reported in minutes. I placed the data in the file named Battery_Data and did histograms. I discovered one of these three data series has some unexpected properties. Import the data into Python. Then view histograms of the data and identify the issue. The web site reported average battery life for each of the three batteries without noting this problem.
- This web site used a sample size $n = 8$. Is this large enough to be comfortable doing a hypothesis test relying on the Central Limit Theorem?

1.9 Statistical Quality Control

Assuring product quality is a central objective of any enterprise. The remainder of this chapter is devoted to quality control. Before turning to the analytics, I thought you might be interested in a brief historical note related to statistical quality control. As you may know, George Westinghouse was one of the most remarkable inventors and managers in the history of the United States, and, indeed, in the history of the world. In Pittsburgh, over the course of his career, he and the companies he created obtained 360 patents! Moreover, he was revered by his employees. When your time permits, be sure to walk over to see the Westinghouse Memorial which is in Schenley Park just across from the northeast corner of the Carnegie Mellon campus. This memorial was created by donations from more than 55,000 employees of companies that he created. If you seek a role model for what you might hope to accomplish as a manager and a human being, you would be hard pressed to find a better role model than George Westinghouse.

I highlight George Westinghouse here because he was among the earliest, if not the earliest, to use statistical quality control, doing so in the 1890's. "Westinghouse wanted all products that bore

his name to maintain the highest standard of quality and reliability. To that end, he introduced rigorous testing and quality control practices at every step of the manufacturing process. Inspectors checked all incoming material to ensure that it met specifications and performance objectives. **Random samples** of finished products were evaluated to make sure they met performance and appearance standards. The entire lot could be rejected if the sample failed. Customer site inspectors were welcomed to do their own observations and evaluations." [Source: William R Huber, **George Westinghouse: Powering the World**, McFarland and Company, Publishers, 2022.] The company motto was "You can be sure if its Westinghouse". Note the use of random sampling, which I emphasized in bold above.

Many products and services are provided in settings in which it is desirable to monitor quality control. The outcome being monitored will depend on the product or service the company is providing.

- Is the correct amount of a product being placed in each package?
- Are defect rates at an acceptable level?
- Are customer satisfaction ratings meeting the expected standard?
- Are call center wait times being kept at an acceptable level?

We now investigate how statistical quality control methods can be employed to address these kinds of quality issues.

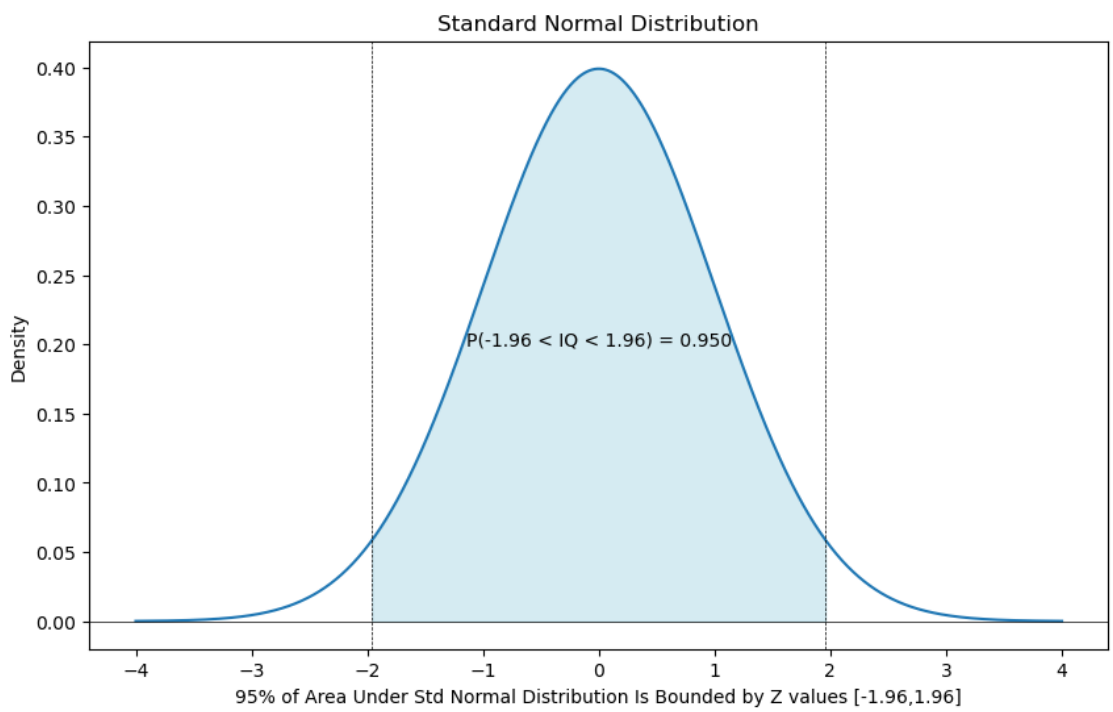
In the analyses that follows, we will be using the normal distribution which was introduced earlier in this chapter. It is standard to use Z to denote a point on the horizontal axis of the normal distribution. $Z_{\alpha/2}$ denotes the value such that $\alpha/2$ of the area under the normal distribution lies to the left of $-Z_{\alpha/2}$ and $\alpha/2$ lies to the right of $+Z_{\alpha/2}$. For example, suppose $\alpha = .05$. Recall that we can find the corresponding Z value using the command below. We obtain $Z_{.05/2} = 1.96$. Thus, 95% of the area under the normal distribution lies in the interval [-1.96,1.96]. This is portrayed in the plot below. Equivalently, we can say that 2.5% of the area lies the left of -1.96 and 2.5% lies to the right of 1.96. This reminds us of the rule of thumb discussed earlier: Approximately 95% of the area under the normal distribution lies within 2 standard deviations of the mean.

[ ]:
```
norm.ppf(0.975)
```

[ ]:
```
np.float64(1.959963984540054)
```

[ ]:
```
s = 3.211915
SE = 1
mu0 = 0
lb = mu0 - 1.96 * SE
ub = mu0 + 1.96 * SE

# Generate data for the plot
x = np.linspace(-4, 4, 1000)
hx = norm.pdf(x)
y = mu0 + x * SE

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(y, hx, label="Standard Normal Distribution")
```

```
plt.fill_between(y, hx, where=(y >= lb) & (y <= ub), color="lightblue", alpha=0.
↪5)
plt.xlabel(
    "95% of Area Under Std Normal Distribution Is Bounded by Z values [-1.96,1.
↪96]"
)
plt.ylabel("Density")
plt.title("Standard Normal Distribution")
plt.axhline(0, color="black", linewidth=0.5)
plt.axvline(lb, color="black", linestyle="--", linewidth=0.5)
plt.axvline(ub, color="black", linestyle="--", linewidth=0.5)

# Calculate the area under the curve between lb and ub
area = norm.cdf(ub) - norm.cdf(lb)
result = f"P({lb:.2f} < IQ < {ub:.2f}) = {area:.3f}"
plt.text(mu0, max(hx) / 2, result, horizontalalignment="center")

# Show the plot
plt.show()
```



1.9.1 Statistical Quality Control for Continuous Quantities

Above we reviewed the concept of an acceptance interval. A key use of the acceptance interval is in statistical quality control. It is useful for you to know that Statistical Quality Control is sometimes

called Statistical Process Control. Let's work through an example. The θ&θ candy company wishes to fill each of its bags of candy with μ=25 ounces of candy. This is a continuous quantity. The θ&θ company knows that its package-filling machines operate with some degree of error. Based on past experience, the company has determined that its machines operate with a standard deviation of σ=.3 when the machines are working properly. The company wishes to implement a quality control system that will detect when a machine is not operating sufficiently close to the desired quantity, μ=25, or has greater than the expected variability, σ=.3.

The approach to statistical quality control exploits the central limit theorem. Let $\overline{X}$ be the mean of a sample of n bags of candy produced by the package-filling machine. The sample size, n is chosen to be large enough that we can rely on the Central Limit Theorem. In particular, the sample is large enough that we can treat $\overline{X}$ as a draw from a normal distribution with mean μ and standard deviation $\frac{\sigma}{\sqrt{n}}$. Recall that we used the t-distribution in hypothesis testing because the variance was unknown and needed to be estimated. In our quality control analysis, the variance σ^2 is given. Hence, we use the normal distribution for our acceptance interval rather than the t-distribution. If the machine is working properly, $\overline{X}$ lies in the acceptance interval with probability $(1-\alpha)$. In symbols, the acceptance interval is: $[\mu - Z_{\alpha/2}\frac{\sigma}{\sqrt{n}}, \mu + Z_{\alpha/2}\frac{\sigma}{\sqrt{n}}]$

1.9.2 Plotting The Control Chart

In the tprstats package, the control_chart() command is used to plot a control chart for continuous quantities.

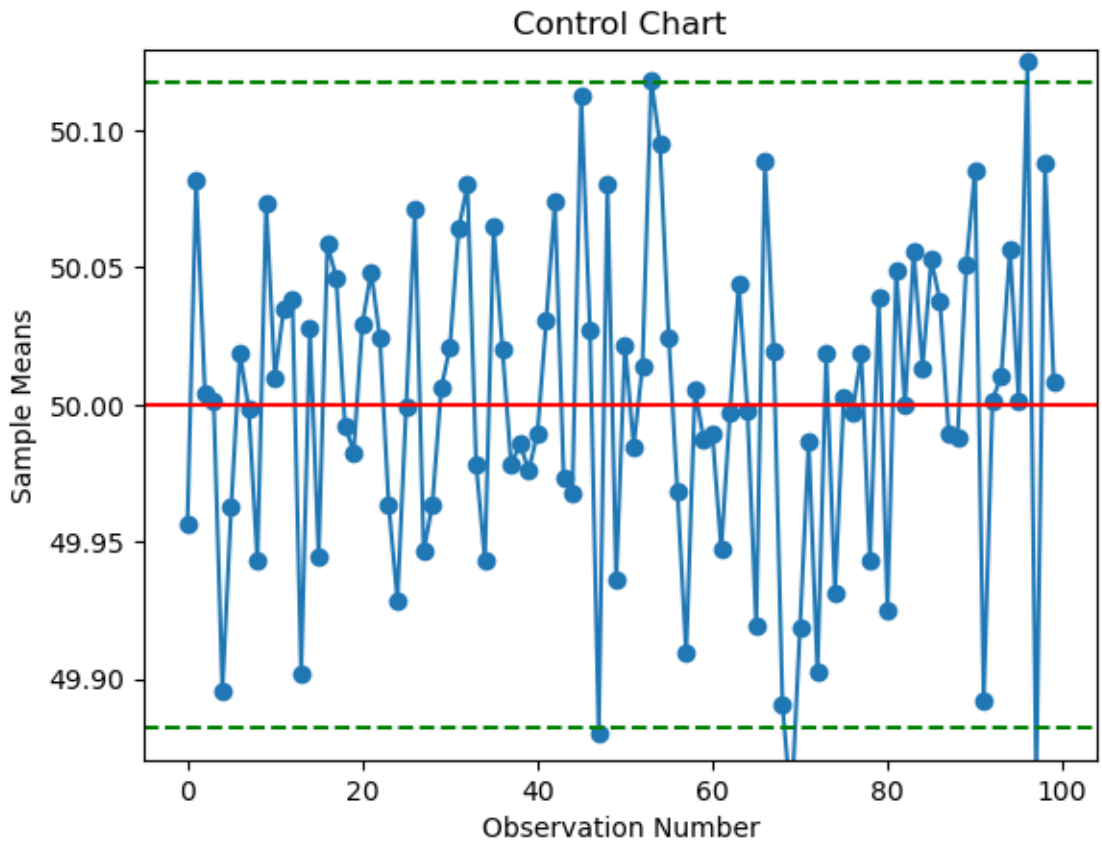
The format of the command is:

`tprstats.control_chart(mu,sig, n, alpha, data).`

- `mu` – The anticipated mean when the production process is working properly.
- `sig` – The anticipated standard deviation when the process is working properly.
- `n` – The number of observations per sample.
- `alpha` – The significance level.
- `data (Dataframe)` – The dataframe to be plotted. There are five entries in this command.

The θ&θ Candy Company weighs a sample of 25 bags of candy every 30 minutes. Sample means for 100 samples are in the file Theta_data.xlsx with variable name XBAR. The control chart is plotted using the command below. The first entry in the command is the variable to be plotted. The second and third entries are the target amount of candy per bag, $\mu = 50$ and the anticipated standard deviation $\sigma = .3$. The fourth entry is the number of bags per sample, 25, and the fifth is the chosen $\alpha = .05$. We see that the process is working well. Five observations are on or outside the bounds. This satisfies the expectation that approximately 95% of the sample means will be inside the bounds when the system is working properly.

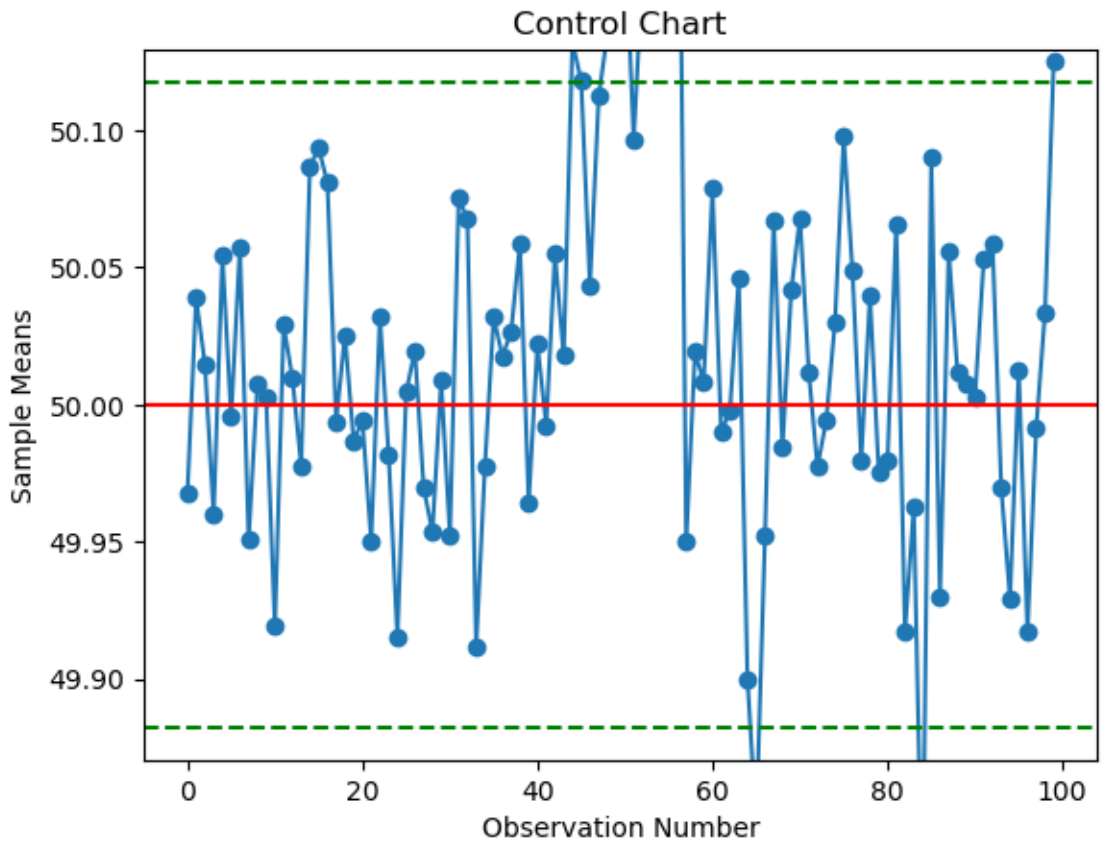
[ ]:
```
tprstats.control_chart(50, 0.3, 25, 0.05, Theta_data["XBAR"])
```



Be sure you understand the logic. Each 30 minutes, a sample of 25 bags is taken. The mean is calculated and saved. The above chart shows a series of 100 sample means that were taken at 30-minute intervals. In practice, the machines operate continuously, and the control chart is continuously updated. Suppose the hundredth sample was taken just now. Then, in 30 minutes, the 101th sample of 25 bags will be taken. The average of those 25 bags will be computed, and that average will be plotted as the 101th observation on the chart. Thirty minutes after that, another sample of 25 will be taken and the mean of the sample will be plotted as the 101th observation.

The following command plots the control chart for XBAR2 from another machine operated by the θ&θ Candy company. This is a case where the machine went out of control for several periods before maintenance was called to correct the problem. If the correction was successful, there is a 95% probability that the mean of the next sample of 25 packages will lie within the control chart bounds.

[ ]:
```
tprstats.control_chart(50, 0.3, 25, 0.05, Theta_data["XBAR2"])
```



For practice, use data in file Theta_new.xlsx. This file is from a new machine, Machine 3, that fills smaller bags of candy. The machine is designed such that, on average, a bag of candy should contain 25 ounces with a standard deviation of .2 (i.e., μ=25, σ=.2). A total of 80 samples have been taken. Each sample is the average of 30 bags of candy. The variable is named XBAR3. Using this information, and the data file, modify the above commands to plot the control chart for this machine with a 98% acceptance interval, i.e., α=.02. Was the machine working as expected over this series of samples?

1.9.3 Control Chart for A Binary Variable

Often the outcome being monitored is a binary outcome. The product is or is not defective. A call to a call center was or was not answered before the customer gave up and hung up. A customer did or did not return the product.

In the tprstats package, the control_chart_binary() command is used to plot a control chart for binary variables.

The format of the command is:

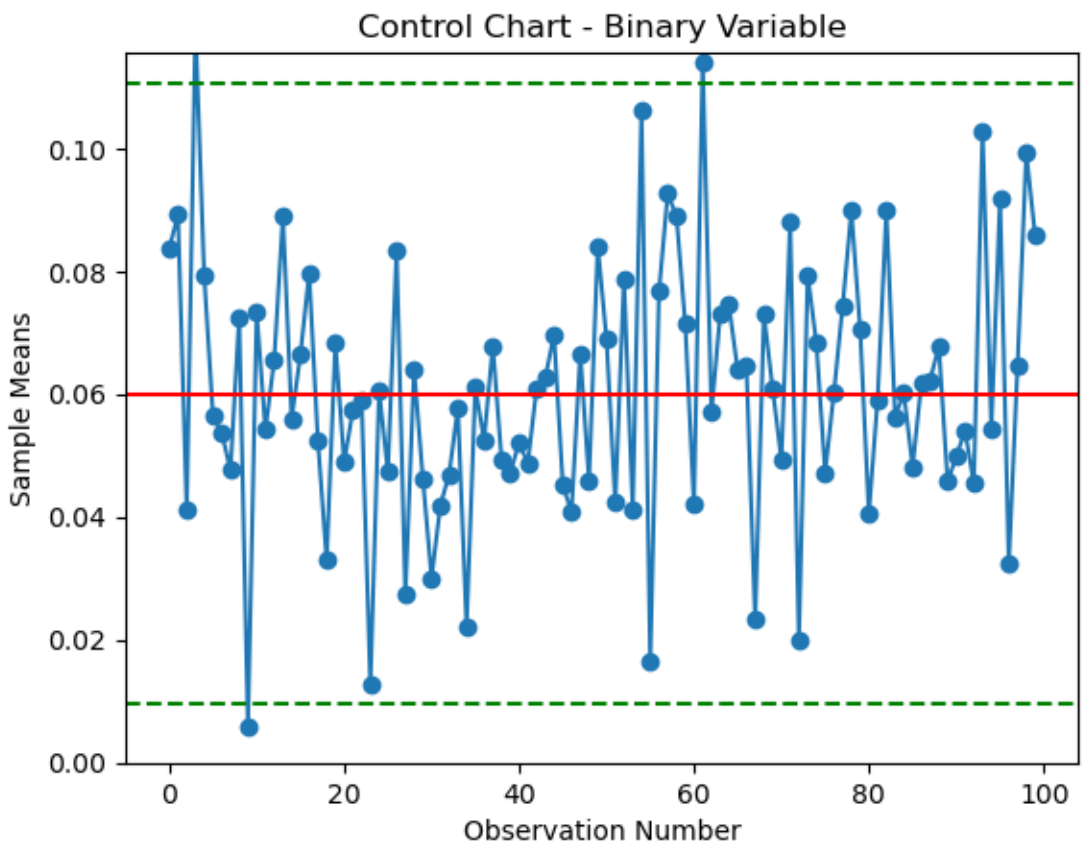
```
tprstats.control_chart_binary(p, n, alpha, data)
```

- p – The target probability for the outcome expressed as a value between 0 and 1.
- n – The number of observations per sample.

- alpha – The significance level.
- data – The dataframe to be plotted.

The following command plots the control chart for monitoring a defect rate. The expected defect rate is .06 when the system is working properly, the sample size is 85, and the significance level is .05. The data are in a file called Defects_data.xlsx, and the variable for defect rate is labeled d_rate. In this application, we are only concerned about the upper bound. It is not a problem if the defect rate is lower than expected. We see from the chart below that the machine was operating according to expectation during the time in which these samples were taken.

[ ]:
```
tprstats.control_chart_binary(0.06, 85, 0.05, Defects_data["d_rate"])
```



When we use a control chart for a binary variable, the number of observations required per sample depends on the target probability. A general rule of thumb is to use the larger of 5/p and 5/(1-p). In our example above, p=.06. Applying the rule of thumb, we obtain 5/.06=83. Thus, the sample size of 85 is large enough to satisfy this rule of thumb.

1.10 Summary

We have covered the Central Limit Theorem, the logic of hypothesis testing, how to test a hypothesis about a population mean, and Statistical Quality Control. Don't be particularly concerned if the Central Limit Theorem still feels a bit elusive to you. Also, this is a substantial amount of material.

Don't worry if you don't have a full grasp of it all at this point. As we go along in the course, you will find that you become increasingly familiar with the concepts and methods and increasingly comfortable with using them. You will also become increasingly comfortable with using Python. By way of review, please return to the beginning of this chapter and remind yourself of the learning objectives. You can use this chapter as a reference when needed to review the details. As a manager, you want to recognize when you have an opportunity to use these methods in practice.

1.11 Appendix: Selecting a Random Sample

In the text we explained that our alum sorted serial numbers of phones in random order. Operationally, how are the serial numbers sorted in random order? Suppose 1000 phones have arrived and the following information for the 1000 phones is in an Excel spreadsheet. Column A has header Serial_Number. Rows A2 through A1001 contain the 1000 serial numbers for the 1000 phones. For the header in column B enter a name, for example, Random. Then, in cell B2, enter the following: =RAND(). Put the pointer on the lower right corner of cell B2 and drag down to cell B1001. This inserts 1000 random numbers in Column B. Mark columns A and B and sort on Random. The top 30 phones will then be a random selection from the 1000 phones.

To illustrate, consider an example with five phones. Suppose we wish to randomly select two phones. The image below shows the serial numbers of the five phones in Column A. The formulas for generating random numbers are shown in Column B.

	A	B
1	Serial_Number	Random
2	1000001	=RAND()
3	1000002	=RAND()
4	1000003	=RAND()
5	1000004	=RAND()
6	1000005	=RAND()

Execute the commands in Column B. This enters a random number in each cell in column B, as shown below. Your random numbers may be different from mine.

	A	B
1	serial_number	Random
2	1000001	0.182298
3	1000002	0.838935
4	1000003	0.08038
5	1000004	0.706635
6	1000005	0.737877

Next, mark both Column A and Column B and sort by Column B. The serial numbers for the five phones are now in random order. We choose the first two phones as our random sample of two phones

	A	B
1	Serial_Number	Random
2	1000003	0.1879832
3	1000004	0.9836655
4	1000001	0.5007076
5	1000005	0.0219034
6	1000002	0.7124493

A quirky feature of Excel is that it automatically chooses a new random number for a cell any time that an action is taken with respect to that cell. That is the reason that the numbers in column B of Figure 7 are not sorted in order. The numbers originally in those cells were replaced during the sort. But, don't worry, the sample has been randomly sorted.

1.11.1 OPTIONAL: Commands Executed when A Control Chart is Plotted

I emphasize that this is optional. Package tprstats contains the code for control_chart() and control_chart_binary(). In case you are curious about what is happening behind the scenes, this optional appendix explains each of the commands that is used when control_chart() is executed.

At the bottom of the page, you will see the commands that are executed when you issue: tprstats.control_chart(50, 0.3, 25, 0.05, Theta_data['XBAR'])

The following explains each line in the code below. I again emphasize that this is optional.

The commands below plot a control chart for XBAR

- mu and sig specify the design mean and standard deviation for the system.
- n is the sample size used in calculating each of the sample means contained in variable XBAR.
- The alpha=.05 value tells Python that we want a 95% acceptance interval.
- m takes on integer values from 1 to the number of observations in data series XBAR. This variable is used on the horizontal axis of the control chart.
- Data are plotted in black, the mean in red, and the acceptance interval in blue.
- Variables ymax and ymin set the range of the vertical axis.

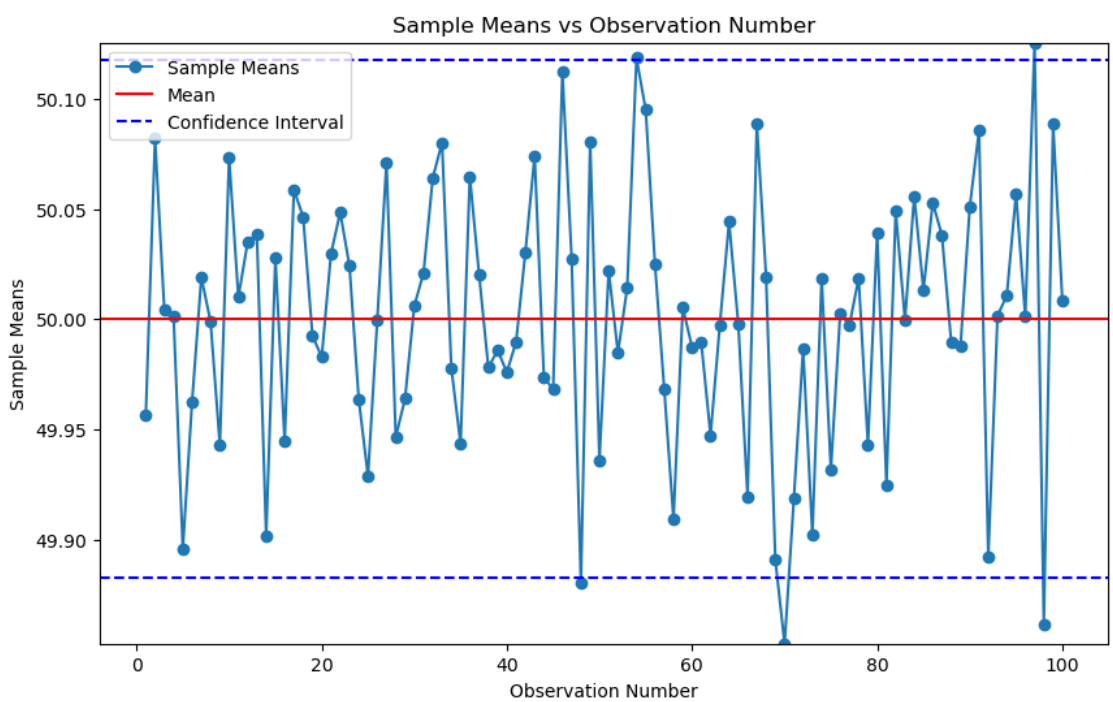
Labeling the Plot

- In the plot command, the label for the horizontal axis is in parentheses following: plt.xlabel()
- The label for vertical axis follows: plt.ylabel()

Telling Python which data file to use:

- The last item in the plot command tells Python which data file to use: data=Theta_data
- The last item in the lines command also tells the data file to use: data=Theta_data

The axhline command is used to add horizontal lines to the graph.

[ ]:
```
mu = 50
sig = 0.3
n = 25
alpha = 0.05

# Sample data (assuming the data are in file Theta_data with variable XBAR)
# Replace this with actual data if available
Theta_data_XBAR = Theta_data["XBAR"]

# Generate observation numbers
m = np.arange(1, len(Theta_data_XBAR) + 1)

# Calculate ymax and ymin
ymax = max(max(Theta_data_XBAR), mu + norm.ppf(1 - alpha / 2) * sig / np.
 ↪sqrt(n))
ymin = min(min(Theta_data_XBAR), mu - norm.ppf(1 - alpha / 2) * sig / np.
 ↪sqrt(n))

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(m, Theta_data_XBAR, "o-", label="Sample Means")
plt.ylim(ymin, ymax)
plt.xlabel("Observation Number")
plt.ylabel("Sample Means")
```

```
plt.title("Sample Means vs Observation Number")

# Add horizontal lines
plt.axhline(mu, color="red", linestyle="-", label="Mean")
plt.axhline(
    mu - norm.ppf(1 - alpha / 2) * sig / np.sqrt(n),
    color="blue",
    linestyle="--",
    label="Confidence Interval",
)
plt.axhline(
    mu + norm.ppf(1 - alpha / 2) * sig / np.sqrt(n), color="blue",␣
 ↪linestyle="--"
)

# Show legend
plt.legend()

# Show the plot
plt.show()
```